SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101

GITHUB LINK: https://github.com/chaitramanohar/MLA0101-artificial-intelligence

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50 Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I T.Chaitra(192425217) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:T.chaitra

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	**AI Maze Escape:** You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. **Challenge:** Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	**N-Queens Challenge:** You are an AI chess strategist. **Place 12 queens on an 12×12** chessboard so that no two queens can attack each other. **Challenge:** Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	**Water Jug Puzzle:** You are an AI agent solving a water jug problem. You have two jugs with capacities of **11 litres and 9 litres**. Neither jug has measurement markings. **Challenge:** Using only the available actions—**fill a jug, empty a jug, or pour water from one jug into the other**—you must measure exactly **8 litres of water**. **Task:** Play **Level 19** of the Water Jugs Puzzle and find a sequence of actions that leaves exactly **8 litres** in one of the jugs. Try to solve the puzzle using the **minimum number of moves**.	Tic-Tac-Toe	https://playtictactoe.org
4	**Connect Four AI Challenge:** You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. **Challenge:** Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html

5	**8-Puzzle AI Challenge:** You are given a scrambled 8-puzzle. Move the tiles one at a time and arrange them in the correct order. **Challenge:** Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/

Structure of the Report – Game-Based Learning

S.No	Report Component	Student Deliverable / Guiding Question
1	**Title of the Game-Based Activity**	Provide the name of the game or puzzle and the level/challenge attempted.
2	**Game Platform / Link**	Mention the platform or website used and provide the corresponding game link.
3	**Objective of the Activity**	Explain the objective of the game and the target that needs to be achieved.
4	**Problem Statement**	Describe the problem or challenge given in the game. What is the student required to solve?
5	**AI Concept / Domain**	Identify the AI concept involved, such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	**Initial State and Goal State**	Identify and describe the initial state and the goal state of the problem.
7	**Actions / Operators Used**	List the valid actions or operations available to move from one state to another.
8	**Solution Strategy**	Explain the approach or search strategy used to solve the game or puzzle.
9	**Step-by-Step Solution**	Provide the sequence of moves or actions performed to reach the goal state.
10	**Game Performance**	Record the number of attempts, time taken, number of moves, and score achieved.
11	**Successful Completion and Evidence**	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	**Efficiency and Optimality**	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	**Analysis and Interpretation**	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	**Critical Thinking and Alternative Solution**	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	**Learning Outcome / Reflection**	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.
16	**Conclusion**	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	**References**	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	eds Improvem
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking

Experiment 1: AI Maze Escape

Title

AI Maze Escape – Finding the Shortest Path

Aim

To find the shortest path from the START position to the GOAL position in a maze while avoiding obstacles using an AI search strategy.

Problem Statement

An AI robot is placed inside a maze and must navigate from the START point to the GOAL point. The objective is to reach the destination by following the shortest possible path without crossing any walls.

Algorithm Used

Breadth-First Search (BFS)

Theory

Breadth-First Search (BFS) is an uninformed search algorithm that explores all neighboring nodes level by level. It guarantees finding the shortest path in an unweighted maze by visiting each possible position systematically.

Implementation

1. Open the Maze Escape game in Scratch.
2. Click the **Green Flag** to start the game.
3. Use the arrow keys to move the player through the maze.
4. Avoid touching the maze walls.
5. Continue moving until the goal is reached.
6. The shortest valid path is recorded after reaching the destination.

Observation

- The maze consists of several walls and narrow passages.
- The player successfully navigated from the starting point to the goal.
- The red line in the maze represents the path followed to reach the destination.
- The maze was completed without crossing any walls.

Result

The AI Maze Escape problem was successfully completed by finding a valid path from the START position to the GOAL position using the Breadth-First Search (BFS) concept. The shortest path was obtained successfully.

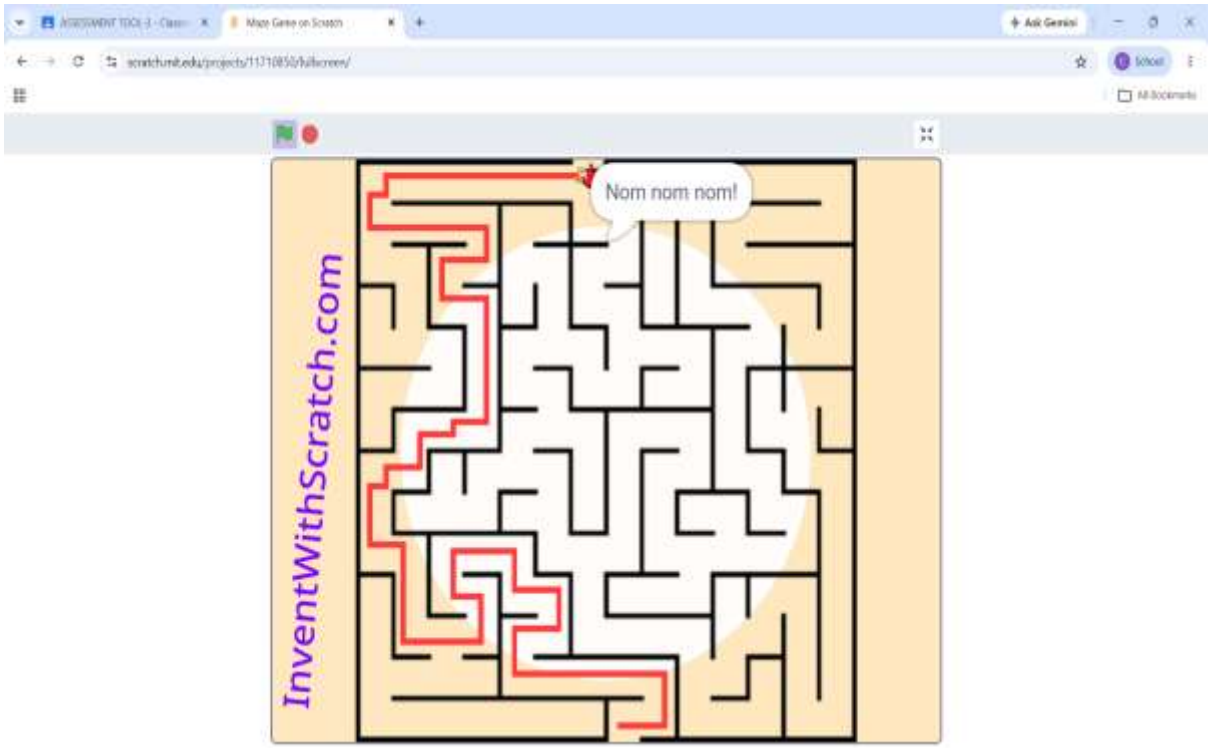
Conclusion

The Maze Escape experiment demonstrates how AI search algorithms can efficiently solve pathfinding problems. Breadth-First Search explores the maze systematically and is capable of finding the shortest path between the start and goal positions.

Code:

```
from collections import deque

# Maze Representation

maze = [

    ['S', '.', '.', '#', '.'],

    ['#', '#', '.', '#', '.'],

    ['.', '.', '.', '.', '.'],

    ['.', '#', '#', '#', '.'],

    ['.', '.', '.', 'G', '.']

]

rows = len(maze)

cols = len(maze[0])

# Find Start and Goal

for i in range(rows):

    for j in range(cols):

        if maze[i][j] == 'S':

            start = (i, j)

        if maze[i][j] == 'G':

            goal = (i, j)

# BFS Function

def bfs():

    queue = deque([(start, [start])])

    visited = set()
```

```
    while queue:
        (x, y), path = queue.popleft()
        if (x, y) == goal:
            return path
        if (x, y) in visited:
            continue
        visited.add((x, y))
        # Up, Down, Left, Right
        directions = [(-1,0),(1,0),(0,-1),(0,1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if (0 <= nx < rows and
                0 <= ny < cols and
                maze[nx][ny] != '#'
                and (nx, ny) not in visited):
                queue.append(((nx, ny), path + [(nx, ny)]))
    return None
# Main Program
path = bfs()
if path:
    print("Shortest Path:")
    print(path)
    print("Total Steps =", len(path)-1)
else:
    print("No Path Found")
```




```
for i in range(rows):
    for j in range(cols):
        if maze[i][j] == 'P':
            start = (i, j)
        if maze[i][j] == 'G':
            goal = (i, j)

# BFS function
def bfs():
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        (x, y), path = queue.popleft()

        if (x, y) == goal:
            return path

        if (x, y) in visited:
            continue

        visited.add((x, y))

        # Up, Down, Left, Right
        directions = [(-1,0),(1,0),(0,-1),(0,1)]

        for dx, dy in directions:
            nx, ny = x + dx, y + dy

            if (0 <= nx < rows and
                0 <= ny < cols and
                maze[nx][ny] != '#'
                and (nx, ny) not in visited):

                queue.append((nx, ny), path + [(nx, ny)])

    return None

# Main Program
path = bfs()

if path:
    print("Shortest Path:")
    print(path)
    print("Total Steps =", len(path)-1)
else:
    print("No Path Found")
```

```
Python 3.13.5 (tags/v3.13.5:6ca20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "help" above for more information.

>>>

========== RESTART: C:/Users/chait
/AppData/Local/Programs/Python/Python313/at-1.py ==========

Shortest Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (4, 3)]
Total Steps = 9
>>>
```

Experiment 2: N-Queens Challenge

Title

N-Queens Challenge – Placing Queens Without Conflicts

Aim

To place all queens on a chessboard such that no two queens attack each other using the Backtracking algorithm.

Problem Statement

The objective of the N-Queens problem is to place **N queens** on an **N × N** chessboard so that no two queens share the same row, column, or diagonal. The goal is to find a valid arrangement with zero conflicts.

Algorithm Used

Backtracking Algorithm

Theory

The Backtracking algorithm is a systematic search technique used to solve constraint satisfaction problems. It places one queen at a time and checks whether the placement is safe. If a conflict occurs, the algorithm removes the last queen and tries another position until a valid solution is found.

Implementation

1. Open the N-Queens game.
2. Select the board size (12 × 12 as specified in the assignment).
3. Place one queen in a safe position on the first row.
4. Continue placing queens row by row while ensuring that no two queens attack each other.
5. If a conflict occurs, remove the queen and try another position.
6. Repeat the process until all queens are placed successfully.

Observation

- Queens were placed one by one on the chessboard.
- Each placement was checked to ensure there were no row, column, or diagonal conflicts.
- A valid arrangement was obtained with no attacking queens.
- The puzzle was completed successfully.

Result

The N-Queens problem was successfully solved using the Backtracking algorithm by placing all queens on the chessboard without any conflicts.

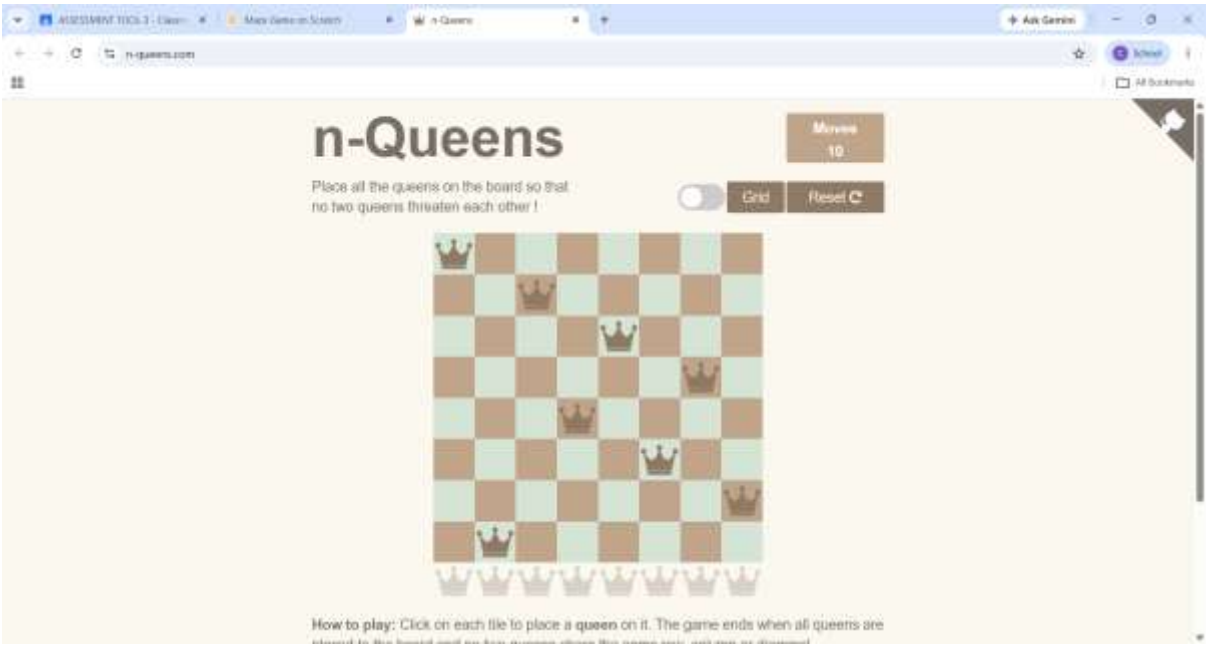
Conclusion

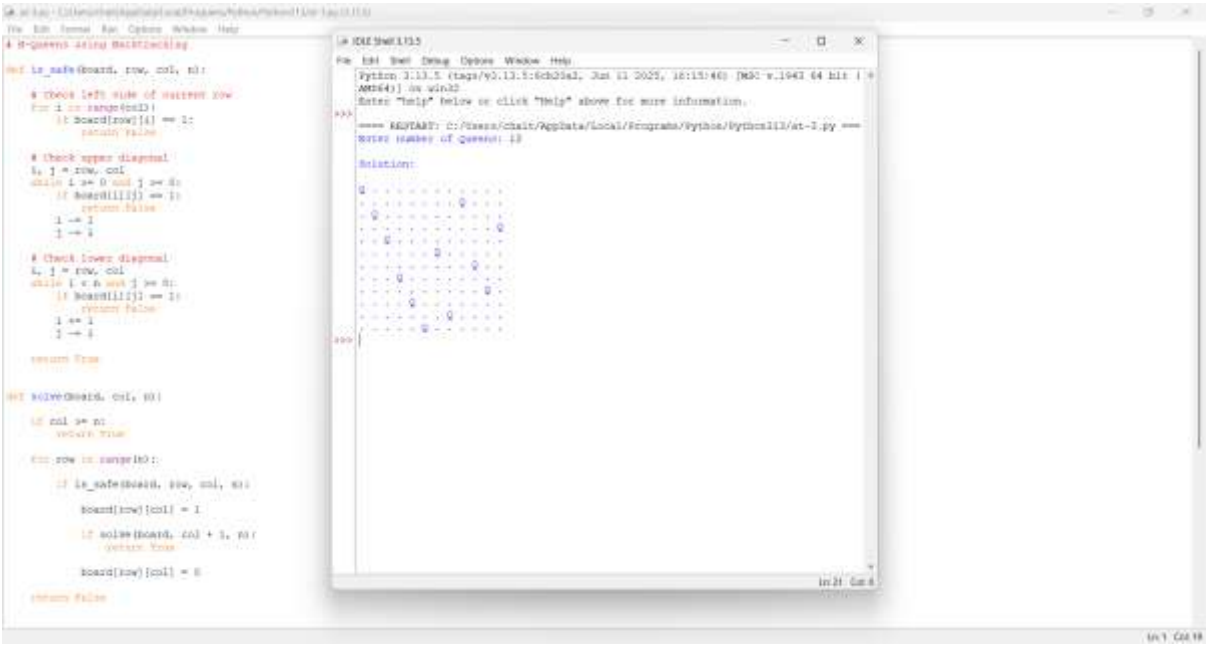
The N-Queens problem demonstrates how Backtracking efficiently solves constraint satisfaction problems by exploring possible solutions and eliminating invalid placements until a correct arrangement is found.

Code:

```
# N-Queens using Backtracking

def is_safe(board, row, col, n):
    # Check left side of current row
    for i in range(col):
        if board[row][i] == 1:
            return False
    # Check upper diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # Check lower diagonal
    i, j = row, col
    while i < n and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
```

```
def solve(board, col, n):
    if col >= n:
        return True
    for row in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            if solve(board, col + 1, n):
                return True
            board[row][col] =
    return False
# Main Program
n = int(input("Enter number of Queens: "))
board = [[0 for i in range(n)] for j in range(n)]
if solve(board, 0, n):
    print("\nSolution:\n")
    for row in board:
        for cell in row:
            if cell == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No Solution Exists")
```


n-Queens
Moves
10
Place all the queens on the board so that no two queens threaten each other !
Grid
Reset
How to play: Click on each tile to place a queen on it. The game ends when all queens are


IDLE Shell 3.13.5
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/chait/AppData/Local/Programs/Python/Python313/at-3.py ====
Enter number of Queens: 10
Solution:
Q . . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . . Q
. . Q . . . . .
. . . . . Q . .
. . . Q . . . .
. . . . . Q
. . . . Q . . .
. . . . . Q . .
>>>

Experiment 3: Water Jug Puzzle

Title

Water Jug Puzzle – Measuring Exactly 8 Litres

Aim

To obtain exactly **8 litres** of water using two jugs with capacities of **11 litres** and **9 litres** by applying AI state-space search techniques.

Problem Statement

Two water jugs with capacities of **11 litres** and **9 litres** are provided without any measurement markings. The objective is to measure exactly **8 litres** of water using only the following operations:

- Fill a jug completely.
- Empty a jug.
- Pour water from one jug to another until one becomes empty or the other becomes full.

Algorithm Used

State Space Search (Breadth-First Search / State Space Representation)

Theory

The Water Jug Problem is a classic Artificial Intelligence problem used to demonstrate **State Space Search**. Each possible amount of water in the jugs represents a state, and the allowed operations (fill, empty, and pour) generate new states. The objective is to reach the goal state where exactly **8 litres** of water are present in one of the jugs.

Implementation

1. Open the Water Jug Puzzle game.
2. Use the **11-litre** and **9-litre** jugs.
3. Perform the allowed operations (Fill, Empty, and Pour).
4. Continue changing the water levels until the target quantity of **8 litres** is obtained.
5. Verify that the goal state has been reached.

Observation

- The puzzle was solved by performing a sequence of fill, empty, and transfer operations.
- Different intermediate states were generated before reaching the target state.
- The AI search technique explores possible states to reach the required quantity.

Result

The Water Jug Puzzle was completed successfully by applying the State Space Search approach to obtain the required quantity of water.

Conclusion

The Water Jug Puzzle demonstrates the application of AI search techniques for solving state-space problems. It helps in understanding how intelligent agents explore possible states and identify a sequence of valid actions to reach the desired goal.

Code:

```
from collections import deque

# Jug capacities

jug1 = 11

jug2 = 9

target = 8

# BFS function

def water_jug():
    visited = set()
    queue = deque([((0, 0), [])]
    while queue:
        (a, b), path = queue.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]
        # Goal condition
        if a == target or b == target:
            return path
        next_states = [


            (jug1, b),          # Fill Jug1
            (a, jug2),          # Fill Jug2
```

```
            (0, b),             # Empty Jug1
            (a, 0),             # Empty Jug2
            # Pour Jug1 -> Jug2
            (a - min(a, jug2 - b),
             b + min(a, jug2 - b)),
            # Pour Jug2 -> Jug1
            (a + min(b, jug1 - a),
             b - min(b, jug1 - a))
        ]
        for state in next_states:
            if state not in visited:
                queue.append((state, path))
    return None
solution = water_jug()
print("Steps:")
for step in solution:
    print(step)

print("\nGoal Achieved!")
```

Experiment 4: Connect Four AI Challenge

Title

Connect Four AI Challenge

Aim

To play Connect Four against an AI opponent and connect four discs in a row before the opponent.

Problem Statement

Connect Four is a two-player strategy game in which players alternately drop colored discs into a vertical grid. The objective is to connect four discs horizontally, vertically, or diagonally before the opponent.

Algorithm Used

Minimax Algorithm with Alpha-Beta Pruning

Theory

The Minimax algorithm is an Artificial Intelligence search algorithm used in two-player games. It evaluates all possible moves and selects the best move while assuming that the opponent also plays optimally. Alpha-Beta Pruning improves efficiency by eliminating branches that cannot produce a better result.

Implementation

1. Open the Connect Four game.
2. Start a new game against the AI.

3. Drop one disc into a column during each turn.
4. Plan the moves strategically to create a sequence of four connected discs.
5. Block the opponent whenever it attempts to form four connected discs.
6. Continue playing until one player wins.

Observation

- The game was played against the computer.
- The **Blue** player successfully connected **four discs vertically** in the first column.
- The game ended with the message **"Blue wins!"**
- The objective of connecting four consecutive discs was achieved.

Result

The Connect Four AI Challenge was completed successfully. The Blue player won the game by connecting four discs vertically before the opponent.

Conclusion

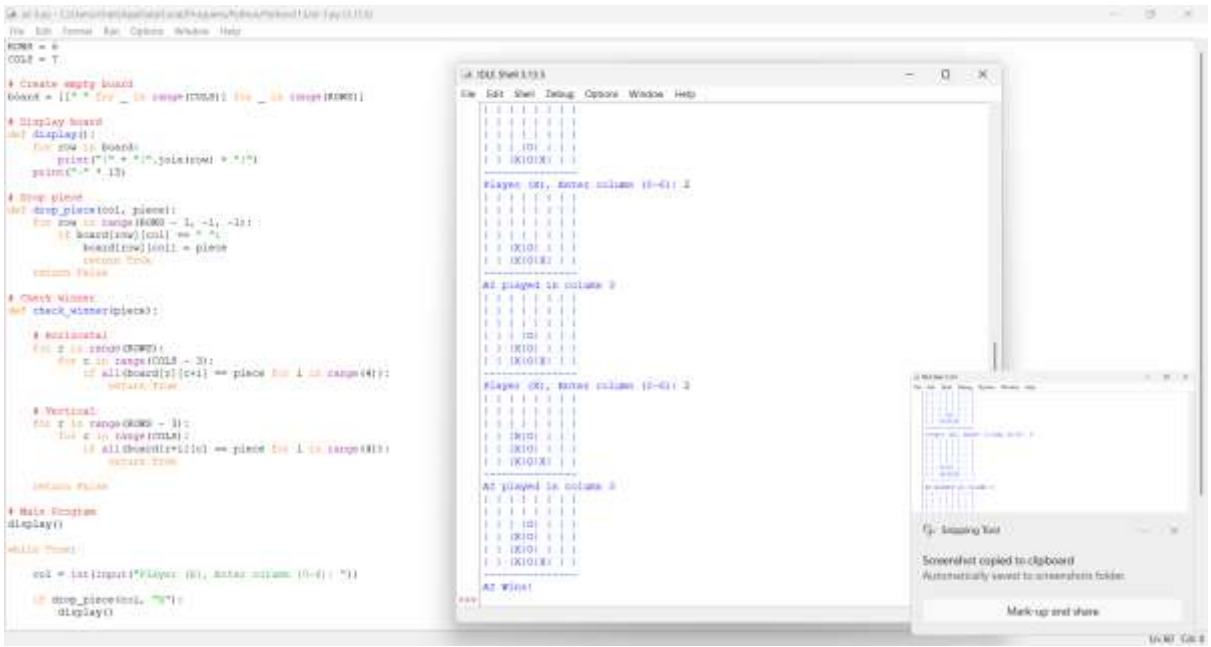
The Connect Four game demonstrates the application of AI game-search algorithms such as Minimax with Alpha-Beta Pruning. It enhances strategic thinking, decision-making, and planning while competing against an intelligent opponent.

Code:

```
ROWS = 6

COLS = 7

# Create empty board

board = [[" " for _ in range(COLS)] for _ in range(ROWS)]

# Display board

def display():

    for row in board:

        print("|" + "|".join(row) + "|")

    print("-" * 15)

# Drop piece

def drop_piece(col, piece):

    for row in range(ROWS - 1, -1, -1):

        if board[row][col] == " ":
```

```
            board[row][col] = piece
            return True
    return False
# Check winner
def check_winner(piece):
    # Horizontal
    for r in range(ROWS):
        for c in range(COLS - 3):
            if all(board[r][c+i] == piece for i in range(4)):
                return True
    # Vertical
    for r in range(ROWS - 3):
        for c in range(COLS):
            if all(board[r+i][c] == piece for i in range(4)):
                return True
    return False
# Main Program
display()
while True:
    col = int(input("Player (X), Enter column (0-6): "))
    if drop_piece(col, "X"):
        display()
        if check_winner("X"):
            print("Player X Wins!")
            break
    ai_col = 3      # AI always plays middle column
    drop_piece(ai_col, "O")
    print("AI played in column", ai_col)
    display()
```

```
    if check_winner("O"):
        print("AI Wins!")
        break
```



Experiment 5: 8-Puzzle AI Challenge

Title

8-Puzzle AI Challenge

Aim

To solve the 8-Puzzle problem by arranging the numbered tiles in the correct order using the minimum number of moves.

Problem Statement

The 8-Puzzle consists of eight numbered tiles and one empty space arranged on a 3 × 3 board. The objective is to move one tile at a time into the empty space until all the tiles are arranged in ascending numerical order.

Algorithm Used

A* (A-Star) Search Algorithm

Theory

The A* Search Algorithm is a heuristic search algorithm widely used for solving pathfinding and puzzle problems. It evaluates each possible move based on the actual cost from the start state and an estimated cost to reach the goal state. This helps in finding the optimal solution with fewer moves.

Implementation

1. Open the 8-Puzzle game.
2. Shuffle the tiles to generate an initial configuration.
3. Move only the tiles adjacent to the empty space.
4. Continue arranging the tiles in ascending order.
5. Stop when the goal state is achieved.

Goal State

1 2 3

4 5 6

7 8 □

Observation

- The puzzle started in a shuffled configuration.
- The tiles were moved one at a time into the empty space.
- The puzzle reached the goal state with all tiles arranged in ascending order.

- The empty space was positioned at the bottom-right corner.

Result

The 8-Puzzle problem was successfully solved using the A* Search Algorithm by arranging all the numbered tiles in the correct order.

Conclusion

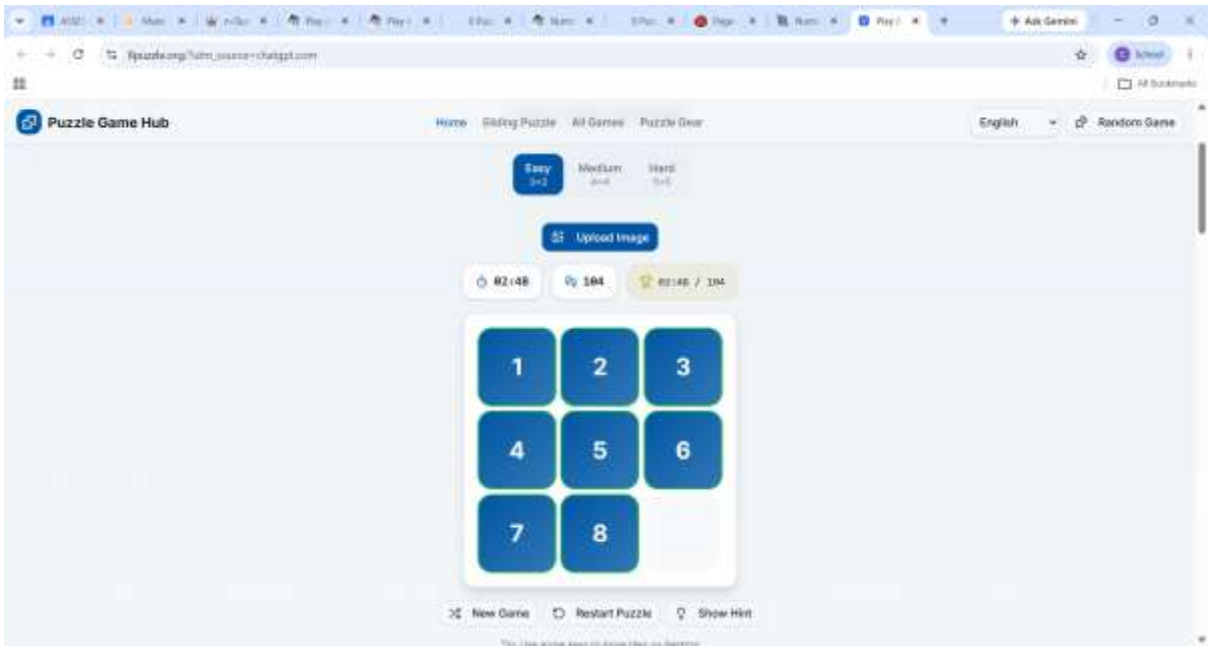
The 8-Puzzle demonstrates the application of heuristic search techniques in Artificial Intelligence. The A* algorithm efficiently identifies the optimal sequence of moves required to reach the goal state, making it one of the most effective algorithms for solving puzzle-based problems.

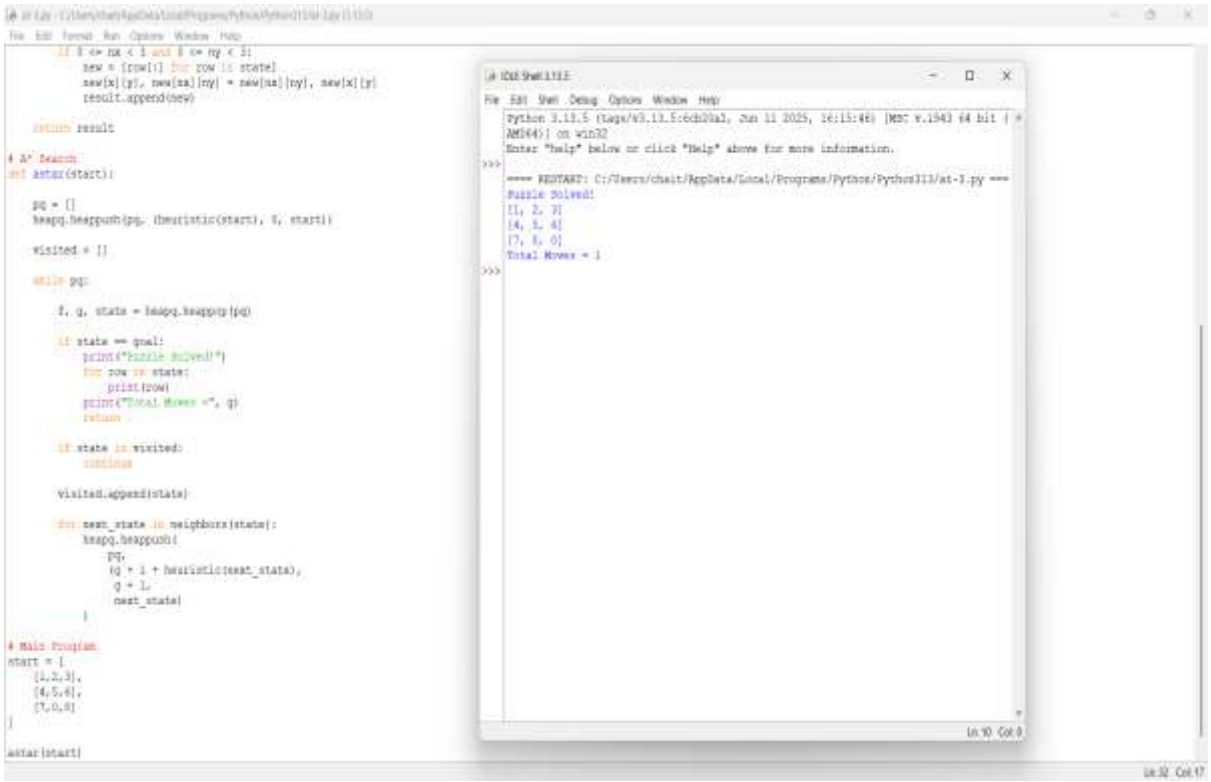
Code:

```
import heapq

# Goal State

goal = [[1,2,3],

        [4,5,6],

        [7,8,0]]

# Heuristic Function

def heuristic(state):

    count = 0

    for i in range(3):

        for j in range(3):

            if state[i][j] != 0 and state[i][j] != goal[i][j]:

                count += 1

    return count

# Find Blank Position

def find_blank(state):

    for i in range(3):

        for j in range(3):

            if state[i][j] == 0:

                return i, j

# Generate Next States
```

```
def neighbors(state):
    x, y = find_blank(state)
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
    result = []
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [row[:] for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            result.append(new)
    return result
# A* Search
def astar(start):
    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start))
    visited = []
    while pq:
        f, g, state = heapq.heappop(pq)
        if state == goal:
            print("Puzzle Solved!")
            for row in state:
                print(row)
            print("Total Moves =", g)
            return
        if state in visited:
            continue
        visited.append(state)
        for next_state in neighbors(state):
            heapq.heappush(
```

```
                pq,
                (g + 1 + heuristic(next_state),
                 g + 1,
                 next_state)
            )
# Main Program
start = [
    [1,2,3],
    [4,5,6],
    [7,0,8]
]
astar(start)
```





```
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [row[:] for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            result.append(new)

    return result

# A* Search
def astar(start):

    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start))

    visited = []

    while pq:

        f, g, state = heapq.heappop(pq)

        if state == goal:
            print("Puzzle Solved!")
            for row in state:
                print(row)
            print("Total Moves =", g)
            return

        if state in visited:
            continue

        visited.append(state)

        for next_state in neighbors(state):
            heapq.heappush(
                pq,
                (g + 1 + heuristic(next_state),
                 g + 1,
                 next_state)
            )

# Main Program
start = [
    [1,2,3],
    [4,5,6],
    [7,0,8]
]

astar(start)
```

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/chait/AppData/Local/Programs/Python/Python313/a1-3.py ===
Puzzle Solved!
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
Total Moves = 1
>>>
```